

Assignment-15 (Lab-15.5)

Course : AI Assisted Coding

Topic: Code Review and Quality

Student Details

Name : Batti Chandan Singh

Hall Ticket : 2303A51515

Batch : 22

Task-01

Final Optimal Prompt

Generate a Python Flask RESTful API for a Student Attendance system.

Implement CRUD operations with endpoints:

GET /attendance, POST /attendance, PUT /attendance/<id>, DELETE /attendance/<id>.

Use JSON request/response format, include error handling for invalid IDs,
and add comments explaining each endpoint.

Code Screenshot

```

6 from flask import Flask, request, jsonify
7
8 app = Flask(__name__)
9
10 attendance = []
11 next_id = 1
12
13
14 @app.route('/attendance', methods=['GET'])
15 def get_attendance():
16     return jsonify(attendance)
17
18
19 @app.route('/attendance', methods=['POST'])
20 def mark_attendance():
21     global next_id
22     data = request.json
23
24     record = {
25         "id": next_id,
26         "student": data.get("student"),
27         "status": data.get("status")
28     }
29
30     attendance.append(record)
31     next_id += 1
32
33     return jsonify(record), 201
34
35
36 @app.route('/attendance/<int:id>', methods=['PUT'])
37 def update_attendance(id):
38     for record in attendance:
39         if record["id"] == id:

```

```

5
6 @app.route('/attendance/<int:id>', methods=['PUT'])
7 def update_attendance(id):
8     for record in attendance:
9         if record["id"] == id:
10             record["status"] = request.json.get("status")
11             return jsonify(record)
12
13     return jsonify({"error": "Record not found"}), 404
14
15
16 @app.route('/attendance/<int:id>', methods=['DELETE'])
17 def delete_attendance(id):
18     for record in attendance:
19         if record["id"] == id:
20             attendance.remove(record)
21             return jsonify({"message": "Record deleted"})
22
23     return jsonify({"error": "Record not found"}), 404
24
25
26 if __name__ == "__main__":
27     app.run(debug=True)

```

Output Screenshot

```

***   * Serving Flask app '__main__'
      * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not
      * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task creates a simple landing page using HTML and CSS. The page includes three main sections: header, hero section, and footer. The header contains the site title and navigation links, while the hero section highlights the main product message and a call-to-action button. CSS is used to style typography, spacing, and layout for a clean and modern design. Responsive principles such as flexible layout and centered content ensure the page adapts across devices. AI assistance helps improve layout alignment and typography choices, making the interface visually appealing. The final design demonstrates basic UI structure commonly used in real-world web applications.

Task-02

Final Optimal Prompt

Generate a Flask REST API for warehouse inventory management.

Implement endpoints:

GET /inventory, POST /inventory, PUT /inventory/<id>, DELETE /inventory/<id>.

Validate stock values so they cannot be negative and return JSON responses with proper error handling.

Code Screenshot

```

inventory = []

@app.route('/inventory', methods=['GET'], endpoint='get_inventory_items')
def get_inventory():
    """Retrieves all inventory items.
    Returns a JSON array of inventory items.
    """
    return jsonify(inventory)

@app.route('/inventory', methods=['POST'], endpoint='add_inventory_item')
def add_item():
    """Adds a new inventory item.
    Expects JSON input with 'name' and 'stock'. 'stock' cannot be negative.
    Returns the newly created item or an error if stock is invalid.
    """
    data = request.json

    if data.get("stock", 0) < 0:
        return jsonify({"error": "Stock cannot be negative"}), 400

    item = {
        "id": len(inventory) + 1,
        "name": data.get("name"),
        "stock": data.get("stock")
    }

    inventory.append(item)
    return jsonify(item)

@app.route('/inventory/<int:id>', methods=['PUT'], endpoint='update_inventory_stock')
def update_stock(id):
    """Updates the stock of an inventory item by ID.
    Expects JSON input with 'stock'. 'stock' cannot be negative.
    Returns the updated item or an error if the item is not found or stock is invalid.
    """

```

```

        inventory.append(item)
        return jsonify(item)

@app.route('/inventory/<int:id>', methods=['PUT'], endpoint='update_inventory_stock')
def update_stock(id):
    """Updates the stock of an inventory item by ID.
    Expects JSON input with 'stock'. 'stock' cannot be negative.
    Returns the updated item or an error if the item is not found or stock is invalid.
    """
    for item in inventory:
        if item["id"] == id:
            new_stock = request.json.get("stock")

            if new_stock < 0:
                return jsonify({"error": "Invalid stock value"}), 400

            item["stock"] = new_stock
            return jsonify(item)

    return jsonify({"error": "Item not found"}), 404

@app.route('/inventory/<int:id>', methods=['DELETE'], endpoint='delete_inventory_item')
def delete_item(id):
    """Deletes an inventory item by ID.
    Returns a success message or an error message if the item is not found.
    """
    for item in inventory:
        if item["id"] == id:
            inventory.remove(item)
            return jsonify({"message": "Item removed"})
    return jsonify({"error": "Item not found"}), 404

```

Output Screenshot

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task implements a theme toggle feature using JavaScript. A button triggers a function that toggles a CSS class on the body element. When the class is applied, the page switches to dark mode with a darker background and lighter text. Removing the class restores the light theme. This approach allows instant visual updates without reloading the page. AI suggestions help ensure proper color contrast for accessibility and user comfort. The implementation is simple but effective, demonstrating how interactive UI features can be built using minimal JavaScript and CSS. Theme toggling is widely used in modern applications to improve user experience.

Task-03

Final Optimal Prompt

Generate a Flask REST API for hotel room booking.
Endpoints: GET /rooms, POST /rooms/book, GET /rooms/<id>, DELETE /rooms/<id>.
Include error handling for unavailable rooms and return JSON responses.

Code Screenshot

```

4 rooms = [
5     {"id": 1, "available": True},
6     {"id": 2, "available": True},
7 ]
8
9 bookings = []
10
11 @app.route('/rooms', methods=['GET'])
12 def view_rooms():
13     return jsonify(rooms)
14
15 @app.route('/rooms/book', methods=['POST'])
16 def book_room():
17     data = request.json
18     room_id = data.get("room_id")
19
20     for room in rooms:
21         if room["id"] == room_id:
22             if not room["available"]:
23                 return jsonify({"error": "Room not available"}), 400
24
25             room["available"] = False
26             booking = {"room_id": room_id}
27             bookings.append(booking)
28             return jsonify(booking)
29
30     return jsonify({"error": "Room not found"}), 404
31
32 @app.route('/rooms/<int:id>', methods=['GET'])
33 def view_booking(id):
34     for booking in bookings:
35         if booking["room_id"] == id:
36             return jsonify(booking)
37
38     return jsonify({"error": "Booking not found"}), 404

```

Output Screenshot

```
1 if __name__ == '__main__':
2     app.run(host='0.0.0.0', port=5000, debug=True)

* Serving Flask app '__main__'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use i
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task builds a simple feedback form for collecting user input. The form includes fields for username, rating, and comments. JavaScript validation checks whether required fields are filled before submission. If the fields are empty, an error message appears dynamically without refreshing the page. When valid input is provided, a success message confirms that the feedback was submitted. AI suggestions help improve layout, usability, and input structure for better user interaction. This approach enhances user experience by providing instant feedback and preventing incomplete submissions. Such validation techniques are commonly used in real-world web applications to ensure data accuracy.

Task-04

Final Optimal Prompt

Create a Flask REST API for an online voting system.

Endpoints: GET /candidates, POST /vote, GET /results.

Ensure one vote per user using in-memory validation and return JSON responses.

Code Screenshot

```
5 candidates = {"Alice": 0, "Bob": 0}
6 voted_users = set()
7
8 @app.route('/candidates', methods=['GET'])
9 def list_candidates():
10     return jsonify(candidates)
11
12 @app.route('/vote', methods=['POST'])
13 def vote():
14     data = request.json
15     user = data.get("user")
16     candidate = data.get("candidate")
17
18     if user in voted_users:
19         return jsonify({"error": "User already voted"}), 400
20
21     if candidate not in candidates:
22         return jsonify({"error": "Invalid candidate"}), 400
23
24     candidates[candidate] += 1
25     voted_users.add(user)
26
27     return jsonify({"message": "Vote recorded"})
28
29 @app.route('/results', methods=['GET'])
30 def results():
31     return jsonify(candidates)
```

Output Screenshot

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task creates a dynamic image gallery using JavaScript. Images are stored in an array, and a variable tracks the currently displayed image index. The next() and prev() functions update the index and load the appropriate image. Navigation buttons allow users to browse through the gallery easily. AI suggestions help optimize the layout for responsive viewing across different screen sizes. The modulo operation ensures smooth looping through images when reaching the first or last item. This approach demonstrates how JavaScript can dynamically update webpage content without refreshing the page, creating a more interactive and engaging user interface.

Task-05

Final Optimal Prompt

Generate a Flask REST API to manage teacher records.

Endpoints: GET /teachers, POST /teachers, PUT /teachers/<id>, DELETE /teachers/<id>.

Ensure JSON responses and proper error handling.

Code Screenshot

```

4 teachers = []
5
6 @app.route('/teachers', methods=['GET'])
7 def get_teachers():
8     return jsonify(teachers)
9
10 @app.route('/teachers', methods=['POST'])
11 def add_teacher():
12     data = request.json
13
14     teacher = {
15         "id": len(teachers) + 1,
16         "name": data.get("name"),
17         "subject": data.get("subject")
18     }
19
20     teachers.append(teacher)
21     return jsonify(teacher)
22
23 @app.route('/teachers/<int:id>', methods=['PUT'])
24 def update_teacher(id):
25     for t in teachers:
26         if t["id"] == id:
27             t["name"] = request.json.get("name")
28             t["subject"] = request.json.get("subject")
29             return jsonify(t)
30
31     return jsonify({"error": "Teacher not found"}), 404
32
33 @app.route('/teachers/<int:id>', methods=['DELETE'])
34 def delete_teacher(id):
35     for t in teachers:
36         if t["id"] == id:
37             teachers.remove(t)
38             return jsonify({"message": "Teacher removed"})
39

```

Output Screenshot

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This task demonstrates how to implement a notification banner on a webpage. The banner appears at the top of the page and can be shown or hidden using JavaScript functions. CSS is

used to style the banner and control its initial visibility. When the user clicks the “Show” button, the banner becomes visible, and clicking “Hide” removes it from view. AI assistance helps ensure proper layout positioning and smooth UI behavior. Notification banners are widely used in modern websites to display alerts, announcements, or promotions. This implementation provides a simple and effective way to manage dynamic content visibility.